

Documentação do Projeto - Etapa 4

Etapa 4 | Lista Dinâmica

Nesta etapa, você deverá implementar uma pilha e uma fila dinâmicas. Para isso, crie um projeto no Eclipse ou na IDE de sua preferência e coloque suas implementações neste projeto. Utilizando as interfaces **Stack** e **Queue**, e a classe **Node**, faça o que se pede:

- A partir da classe **Node** e da interface **Stack**, crie uma classe chamada **LinkedStack**, que implementa **Stack**, ou seja, é uma pilha encadeada.
- A partir da classe **Node** e da interface **Queue**, crie uma classe chamada **LinkedQueue**, que implementa **Queue**, ou seja, é uma fila encadeada.

Arquivo de Teste - PrincipalEtapa4.java

Implemente, em uma classe chamada **PrincipalEtapa4.java**, testes para validar as funcionalidades de **LinkedStack** e **LinkedQueue**:

- **Teste 1:** Verificação de elementos em uma pilha (LinkedStack) com operações de empilhar, desempilhar e verificar o topo.
- **Teste 2:** Verificação de elementos em uma fila (LinkedQueue) com operações de enfileirar, desenfileirar e verificar o primeiro e último elemento.

Código - PrincipalEtapa4.java

```
// Teste 1: Operações com Pilha (LinkedList)
System.out.println("Teste 1: Contagem de Elementos na Pilha (LinkedList)");
LinkedList stack = new LinkedList<>();

// Verifica se a pilha está vazia
System.out.println("Pilha está vazia? " + stack.isEmpty());

// Empilhando elementos
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println("Elemento no topo da pilha: " + stack.top());

// Desempilhando um elemento
System.out.println("Desempilhando: " + stack.pop());
System.out.println("Número de elementos na pilha: " + stack.numElements());
System.out.println("-----");

// Teste 2: Operações com Fila (LinkedList)
System.out.println("Teste 2: Verificação de Elementos na Fila (LinkedList)");
LinkedList queue = new LinkedList<>();

// Verifica se a fila está vazia
System.out.println("Fila está vazia? " + queue.isEmpty());

// Enfileirando elementos
queue.enqueue(100);
queue.enqueue(200);
queue.enqueue(300);
System.out.println("Primeiro da fila: " + queue.front());

// Desenfileirando um elemento
System.out.println("Desenfileirando: " + queue.dequeue());
System.out.println("Número de elementos na fila: " + queue.numElements());

// Verificar último elemento (back)
System.out.println("Último da fila (back): " + queue.back());
System.out.println("-----");
```

Exemplo de Saída - Teste

Abaixo, um exemplo de saída esperada ao rodar os testes:

Saída

Teste 1: Contagem de Elementos na Pilha (LinkedList)

Pilha está vazia? true

Elemento no topo da pilha: 30

Desempilhando: 30

Número de elementos na pilha: 2

Teste 2: Verificação de Elementos na Fila (LinkedList)

Fila está vazia? true

Primeiro da fila: 100

Desenfileirando: 100

Número de elementos na fila: 2

Último da fila (back): 300

Diagrama de Classes

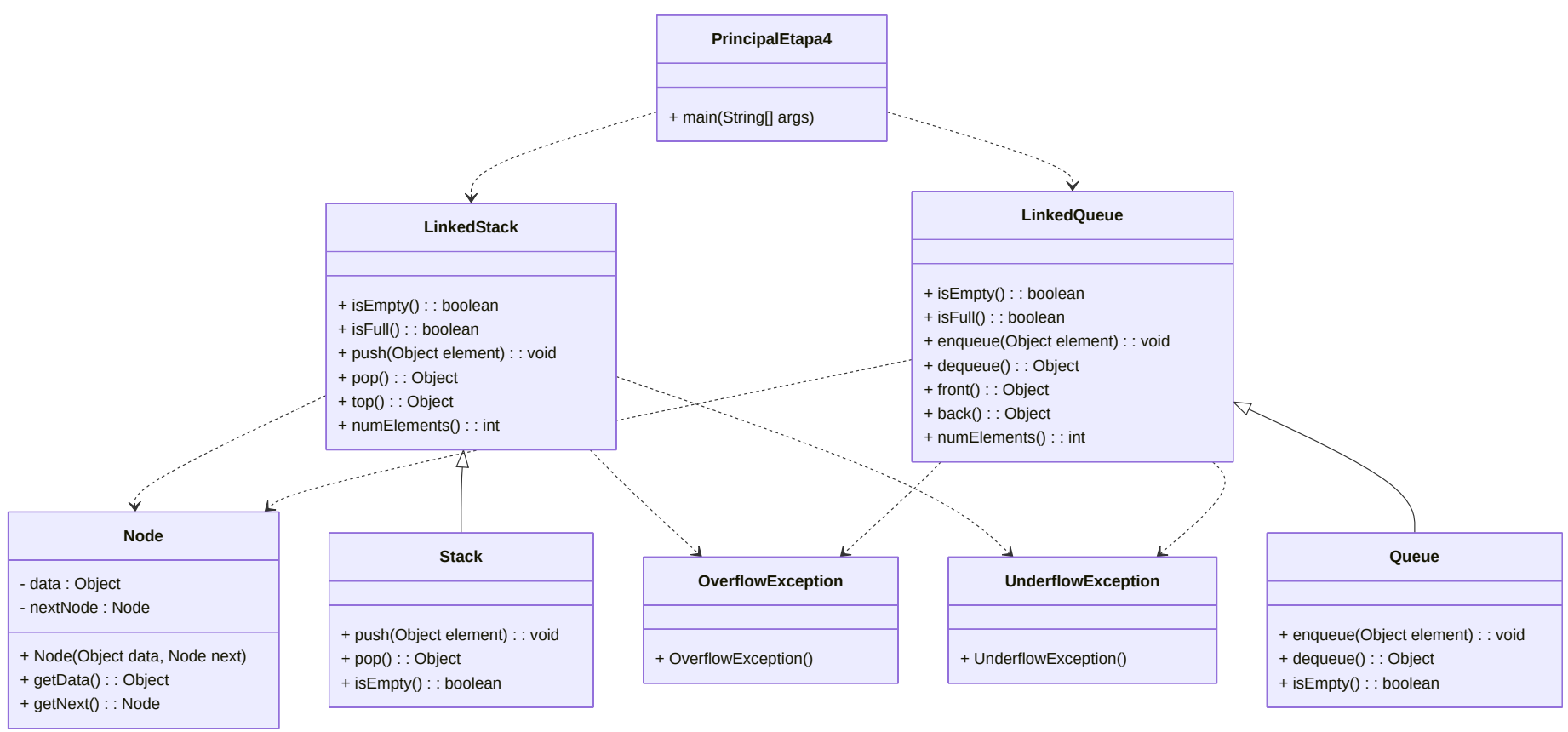
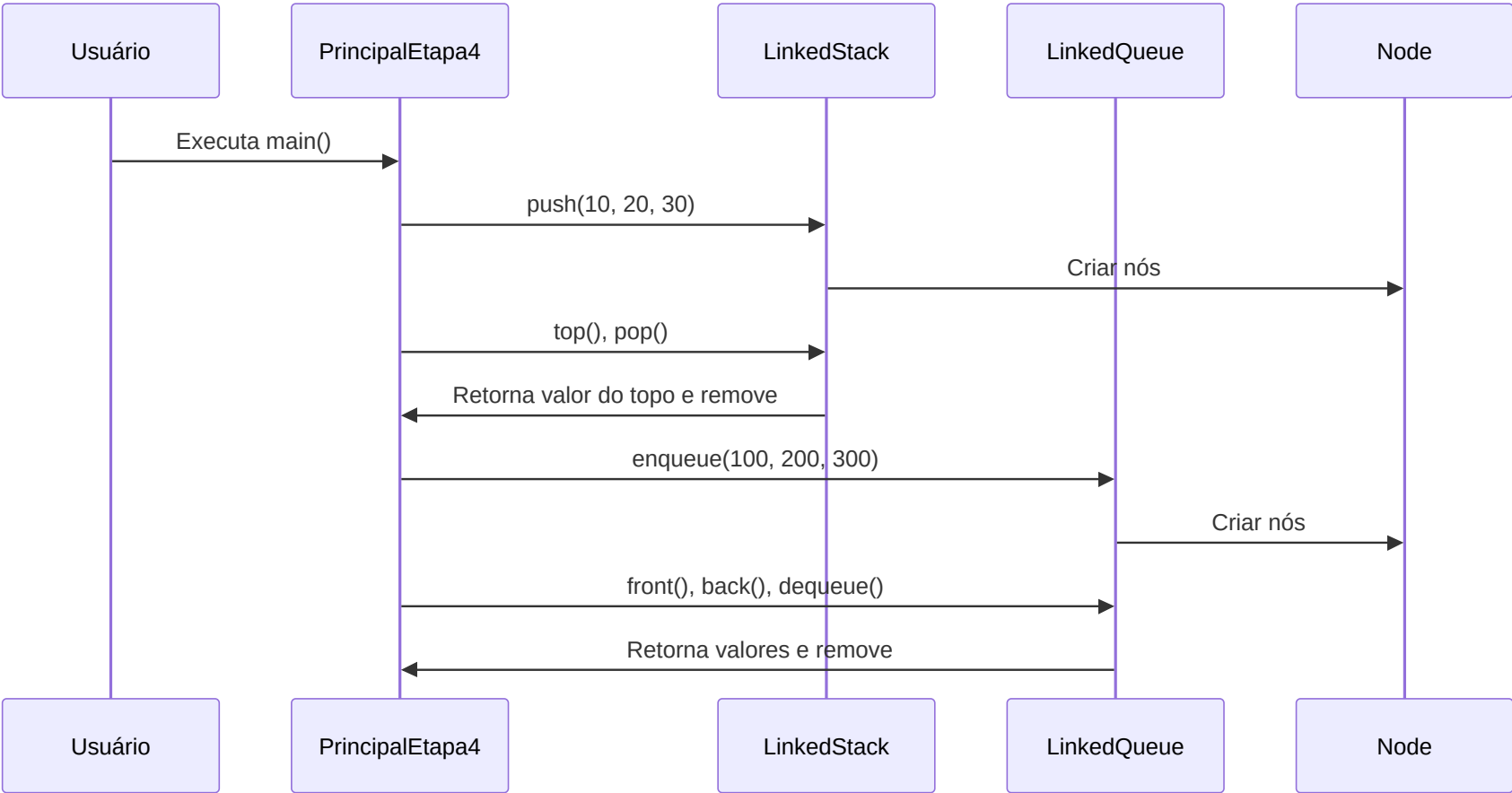


Diagrama de Sequência - Testes Etapa 4



Descrição das Classes e Métodos

Classe PrincipalEtapa4

Descrição: Classe responsável por executar os testes das classes `LinkedStack` e `LinkedQueue`, contendo o método `main`.

Método principal (main):

- Executa dois testes:
 - Operações com pilha (empilhar, desempilhar, verificar topo).
 - Operações com fila (enfileirar, desenfileirar, verificar primeiro e último elementos).

Classe LinkedStack

Descrição: Implementa uma pilha dinâmica usando encadeamento.

Métodos:

- `isEmpty()`: Verifica se a pilha está vazia.
- `isFull()`: Verifica se a pilha está cheia (não aplicável a listas encadeadas).
- `push(E element)`: Adiciona um elemento ao topo da pilha.
- `pop()`: Remove e retorna o elemento do topo da pilha.
- `top()`: Retorna o elemento no topo da pilha sem removê-lo.
- `numElements()`: Retorna o número de elementos na pilha.

Classe LinkedQueue

Descrição: Implementa uma fila dinâmica usando encadeamento.

Métodos:

- `isEmpty()`: Verifica se a fila está vazia.
- `isFull()`: Verifica se a fila está cheia (não aplicável a listas encadeadas).
- `enqueue(E element)`: Adiciona um elemento ao final da fila.
- `dequeue()`: Remove e retorna o primeiro elemento da fila.
- `front()`: Retorna o primeiro elemento da fila sem removê-lo.
- `back()`: Retorna o último elemento da fila sem removê-lo.
- `numElements()`: Retorna o número de elementos na fila.

 Classes de Exceções

OverflowException: Exceção lançada quando uma operação tenta adicionar um elemento a uma pilha ou fila que já está cheia.

UnderflowException: Exceção lançada quando uma operação tenta remover um elemento de uma pilha ou fila que está vazia.